

Architectural Style Justification

Microservices Architecture for the Carpooling & Expense Splitter System.

Reason 1: Matching gets way busier than everything else

Our requirements say the app needs to handle 5,000 people trying to match rides at the same time during rush hour (8–10 AM, 5–7 PM), and still respond in under 3 seconds (NFR-002). But that traffic spike is really only hitting the matching part of the app. Booking, fare splitting, and driver verification don't get nearly as busy. With microservices we can just add more copies of the Ride Matching service during rush hour instead of scaling up the whole app for no reason.

Reason 2: If the payment gateway breaks, it shouldn't break everything else

Booking a ride depends on an outside payment gateway, and outside services fail sometimes, and our own use case flow even has a whole alternate path for when payment gets declined. If we build this as one big connected system, a slow or broken payment gateway could end up freezing other features too. By keeping payment in its own service, a payment problem only affects booking/payment, not matching, verification, or notifications.

Security Advantage

Driver verification handles sensitive stuff, like license and vehicle documents (FR-004, NFR-001). Since it's its own separate service in microservices, we can lock it down with its own database and stricter access rules, instead of it sharing storage with less sensitive stuff like notifications. So even if some other part of the app got hacked, the verification documents would still be protected in their own space.

Performance Benefit

Because each service runs on its own, we can scale up just the Fare Split or Matching services when they're getting hit hard, without touching the rest of the app. This helps us hit the 5-second booking confirmation target from FR-002, since the Booking service is just calling other small, already-running services instead of competing for resources with unrelated parts of the app.